

```

# Datei: infrastructure/languages/translator.py
"""Infrastructure Layer: KI-gestützte Offline-Übersetzung und automatischer Validierungs-Servic

import os
import json
import urllib.request
import requests # <-- DIESE ZEILE HIER DIREKT ERGÄNZEN!
import infrastructure.cfg as cfg

class DynamicLanguageManager:
    """Verwaltet dynamische Sprachvorschläge via Ollama und generiert/validiert Sprach-Vorlage

    def __init__(self, services: dict = None):
        self._services = services
        self.active_languages = cfg.APP_CONFIG.get("supported_languages", ["de", "en"])

    def check_ollama_status(self) -> bool:
        """Prüft schnell und lautlos, ob der lokale Ollama-Server erreichbar ist."""
        import requests
        try:
            # Nutzt die Standard-Ollama-URL aus deiner Konfiguration
            url = getattr(self, "base_url", "http://localhost:11434")
            # Ein kurzes Timeout verhindert, dass die GUI einfriert, falls Ollama hängt
            response = requests.get(url, timeout=15.0)
            return response.status_code == 200
        except Exception:
            return False

    def suggest_translation(self, text: str, target_lang_name: str) -> str:
        """Generiert offline mithilfe des lokalen Ollama-LLMs einen intelligenten Übersetzungs
        model_name = cfg.APP_CONFIG.get("default_model", "llama3")

        if self._services and "biz" in self._services:
            biz = self._services["biz"]
            if hasattr(biz, "_llm") and hasattr(biz._llm, "model_name"):
                model_name = biz._llm.model_name

        if not self.check_ollama_status():
            return text

        # Umstellung auf die schnellere generate-API (identisch zum erfolgreichen Testskript)
        url = "http://localhost:11434/api/generate"

        # Ein kompakter, klarer englisch-deutscher Hybrid-Prompt, den Llama3 perfekt versteht
        prompt = (
            f"You are a professional translator for the historical software framework ClioGrap
            f"Translate the German term '{text}' precisely, scholarly, and extremely compact in
            f"Examples:\n"
            f"- 'Quellenverwaltung' -> 'Source Management' / 'Archivio' / matching terms\n"
            f"- 'Datenbank' -> 'Database'\n"
            f"- 'Hilfe' -> 'Help'\n\n"
            f"CRITICAL RULE: Reply ONLY with the raw translated term or matching short phrase.

```

```

        f"Do not repeat the original text if a translation exists. "
        f"No explanations, no quotes, no conversational text. Only the direct translation.
    )

```

```

payload = {
    "model": model_name,
    "prompt": prompt,
    "stream": False,
    "options": { "temperature": 0.1 }
}

```

```

try:
    # Nutzung der installierten requests-Bibliothek mit robustem 15-Sekunden-Timeout
    response = requests.post(url, json=payload, timeout=15.0)

    if response.status_code == 200:
        result_json = response.json()
        message_content = result_json.get("response", "").strip()
        # Bereinigung von störenden Satzzeichen, falls die KI welche mitsendet
        return message_content.replace("'", '').replace('"', '').replace(".", "")
    else:
        print(f"?? [KI-Fehler] Server antwortet mit Status-Code {response.status_code}")

except requests.exceptions.Timeout:
    print("?? [Ollama-Timeout] Abfrage übersprungen. Das Zeitlimit von 15 Sekunden wur
except Exception as e:
    print(f"?? [KI-Fehler] Abfrage fehlgeschlagen: {e}")

```

```

return text

```

```

def build_new_language_pack(self, target_code: str, target_name: str, i18n_service) -> boo
    """Klont die de.json, übersetzt alle Schlüssel offline via Ollama und speichert sie ab
    source_translations = i18n_service._translations.get("de", {})
    if not source_translations: return False

    new_package = {}
    print(f"? [Sprach-Designer] Starte Offline-KI-Gesamtexport für '{target_name}'...")

    for key, de_text in source_translations.items():
        new_package[key] = self.suggest_translation(de_text, target_name)

    target_file = os.path.join(cfg.JSON_LOCALES_DIR, f"{target_code}.json")
    try:
        with open(target_file, "w", encoding="utf-8") as f:
            json.dump(new_package, f, indent=4, ensure_ascii=False)
        i18n_service._translations[target_code] = new_package
        if target_code not in cfg.APP_CONFIG["supported_languages"]:
            cfg.APP_CONFIG["supported_languages"].append(target_code)
        if {"name": target_name, "code": target_code} not in cfg.AVAILABLE_LANGUAGES:
            cfg.AVAILABLE_LANGUAGES.append({"name": target_name, "code": target_code})
        return True
    except Exception:

```

```
return False
```

```
def validate_and_heal_locales(self, target_code: str, target_name: str, i18n_service) -> i
    """Scannt die Sprachdatei der gewählten Zielsprache und heilt alle identischen/fehlend
    de_dict = i18n_service._translations.get("de", {})

    # 1. Die Übersetzungsdatei der Zielsprache laden (z.B. fr.json oder fa.json)
    target_file = os.path.join(cfg.JSON_LOCALES_DIR, f"{target_code}.json")
    target_dict = {}

    if os.path.exists(target_file):
        try:
            with open(target_file, "r", encoding="utf-8") as f:
                target_dict = json.load(f)
        except Exception:
            target_dict = {}

    repairstatistiken = 0
    print(f"? [KI-Validierung] Starte Überprüfung für Zielsprache: {target_name} ({target_

    # 2. Schleife über alle existierenden deutschen Master-Schlüssel
    for key, de_text in de_dict.items():
        if key in ["app.title", "dialog.language.select.title"]: continue

        # WANN MUSS DIE KI HEILEN?
        # Wenn der Schlüssel in der Zielsprache fehlt ODER noch den deutschen Text enthält
        if key not in target_dict or target_dict[key] == de_text or target_dict[key].strip
            print(f"? [Validierung -> {target_code}] Repariere: '{de_text}'")
            geheiltes_wort = self.suggest_translation(de_text, target_name)

            if geheiltes_wort != de_text:
                target_dict[key] = geheiltes_wort
                repairstatistiken += 1
            else:
                target_dict[key] = de_text

    # 3. Wenn Reparaturen stattfanden, die Datei physisch zurückschreiben
    if repairstatistiken > 0 or not os.path.exists(target_file):
        try:
            with open(target_file, "w", encoding="utf-8") as f:
                json.dump(target_dict, f, indent=4, ensure_ascii=False)
                i18n_service._translations[target_code] = target_dict # RAM aktualisieren
        except Exception as e:
            print(f"? Fehler beim Schreiben der Reparatur-Datei {target_code}.json: {e}")

    return repairstatistiken
```